

UNIVERSITÀ POLITECNICA DELLE MARCHE
FACOLTÀ DI INGEGNERIA
Dipartimento di Ingegneria dell'Informazione
Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione



CHATBOT

ChatBot DataScience

Autori del progetto

Erxhes Dedja - 1119284

Docenti

Prof. Domenico Ursino

ANNO ACCADEMICO 2024-2025

1	Framework RASA	1
1.1	ChatBot	1
1.2	RASA	2
1.3	RASA NLU	2
1.4	RASA CORE	2
2	ChatBot	4
2.1	Introduzione	4
2.2	Ambiente Virtuale	4
2.3	Database	5
2.4	Struttura del progetto	5
2.5	Logica conversazionale e azioni del chatbot	6
2.5.1	Ricerca dei prodotti	6
2.5.2	Risposte e guida alla conversazione	7
3	Funzionalità	8
3.1	Introduzione	8
3.2	Avvio della conversazione	8
3.3	Informazioni sullo store	8
3.4	Prodotti presenti nello store	8
3.5	Informazioni sui prodotti	9
3.6	Riconoscimento robusto dei nomi	9
3.7	Suggerimenti contestuali	9
3.8	Esempio di conversazione completa	10
4	Telegram	11
4.1	Creazione del Bot Telegram	11
4.2	Architettura di collegamento	12
4.3	Configurazione del server Flask	12
4.4	Avvio del server	12
4.5	Esempi di interazione	13
4.5.1	Disponibilità di un prodotto	14
4.5.2	Prezzo	14
4.5.3	Taglie e colore	15
4.5.4	Indisponibilità di un prodotto	15

4.5.5	Filtro per categoria e target	16
4.6	Considerazioni finali	16
Bibliografia		17

Elenco delle figure

1.1	Logo RASA	2
2.1	Struttura del progetto RASA personalizzata	5
3.1	Esempio di dialogo completo: disponibilità, prezzo e colore	10
4.1	Creazione del bot con BotFather	11
4.2	Avvio di ngrok e ottenimento dell'URL pubblico	13
4.3	Avvio del server webhook Flask e registrazione automatica del webhook	13
4.4	Richiesta di disponibilità su Telegram	14
4.5	Richiesta del prezzo	14
4.6	Richiesta delle taglie e del colore	15
4.7	Richiesta di un prodotto non disponibile	15
4.8	Filtro	16

1.1 ChatBot

Un chatbot è un programma informatico che simula e gestisce conversazioni simili a quelle umane, permettendo di interagire con dispositivi digitali in modo naturale e fluido. Può essere basato su regole predefinite o utilizzare l'intelligenza artificiale per rispondere in modo più naturale e adattabile.

Tipologie principali di chatbot:

- **Chatbot Basati su Regole:** I chatbot basati su regole, spesso chiamati clickbot, funzionano seguendo percorsi di conversazione predefiniti. In pratica, offrono agli utenti una serie di opzioni preimpostate tra cui scegliere per domande e risposte. Tuttavia, se un utente pone una domanda al di fuori di queste opzioni pianificate, il bot non è in grado di fornire una risposta.

I chatbot basati su regole sono spesso utilizzati per rispondere a domande frequenti dei clienti, gestire prenotazioni di appuntamenti in modo automatizzato e fornire supporto tecnico di base, offrendo soluzioni a problemi comuni. Questi sono particolarmente adatti per attività in cui le interazioni sono prevedibili e ripetitive, e sono utilizzati in settori come il servizio clienti, il supporto tecnico e le vendite online per automatizzare le risposte a domande comuni e migliorare l'efficienza operativa.

- **Chatbot Basati su Intelligenza Artificiale:** I chatbot basati sull'intelligenza artificiale sono sistemi sofisticati che sfruttano il machine learning e il processamento del linguaggio naturale (NLP) per interpretare e rispondere alle richieste degli utenti. A differenza dei chatbot che seguono regole fisse, questi chatbot possono gestire conversazioni più articolate e adattarsi a diversi tipi di domande e situazioni.

Una delle caratteristiche distintive di questi chatbot è la loro capacità di evolversi e perfezionarsi nel tempo. Grazie ai feedback ricevuti dagli utenti e all'elaborazione di nuovi dati, possono affinare le loro risposte e migliorare l'efficacia delle interazioni. Questo processo di apprendimento continuo li rende altamente adattabili e in grado di gestire una vasta gamma di scenari.

1.2 RASA

Rasa è un framework open-source dedicata allo sviluppo di chatbot e assistenti virtuali conversazionali. Rasa si distingue per la sua capacità di gestire interazioni complesse e di personalizzare le risposte grazie all'impiego di tecnologie avanzate come l'apprendimento automatico e il processamento del linguaggio naturale (NLP). A differenza di altre soluzioni che offrono pacchetti preconfezionati, Rasa mette a disposizione degli sviluppatori un'elevata flessibilità e controllo, consentendo loro di creare chatbot su misura, capaci di adattarsi perfettamente alle esigenze specifiche dell'utente. L'architettura di Rasa si compone di due moduli chiamati Rasa NLU (Natural Language Understanding) e Rasa Core.

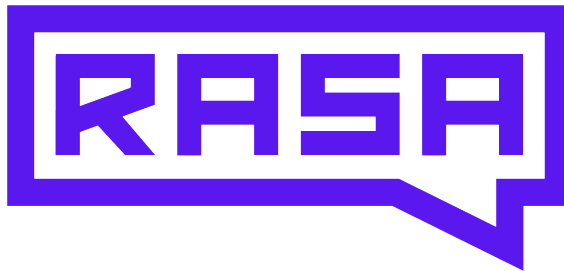


Figura 1.1: Logo RASA

1.3 RASA NLU

Rasa NLU (Natural Language Understanding) è la componente di Rasa che si occupa di interpretare il linguaggio naturale. Si occupa di interpretare il testo inserito dagli utenti, identificando intenti ed estraendo entità, informazioni fondamentali per rispondere in modo adeguato alle richieste.

Rasa NLU sfrutta algoritmi di apprendimento automatico per comprendere il linguaggio dell'utente, classificando il testo secondo diversi intenti e identificando informazioni chiave chiamate entità. Gli intenti rappresentano ciò che l'utente intende fare, come richiedere informazioni, effettuare un acquisto, o fissare un appuntamento. Le entità, invece, sono i dettagli importanti presenti nella frase, come date, orari, luoghi e oggetti.

La classificazione degli intenti e l'estrazione delle entità avvengono mediante modelli di machine learning addestrati con dati di esempio. Questo approccio consente a Rasa NLU di riconoscere le diverse modalità con cui gli utenti possono esprimere le stesse intenzioni, migliorando così la precisione e la versatilità del sistema.

1.4 RASA CORE

Rasa Core è utilizzata per gestire il flusso delle conversazioni nei chatbot. A differenza di Rasa NLU, che si occupa di comprendere il linguaggio naturale degli utenti identificando intenti ed entità, Rasa Core è responsabile della gestione della logica della conversazione, decidendo come il chatbot deve rispondere in base al contesto e alla sequenza delle interazioni.

Rasa Core utilizza una rete neurale di apprendimento per rinforzo per prevedere la prossima azione del chatbot. In pratica, ogni volta che l'utente invia un messaggio, Rasa Core valuta l'intero contesto della conversazione fino a quel punto e decide quale dovrebbe essere la prossima mossa del chatbot. Questa previsione si basa su dati di addestramento che

includono esempi di conversazioni reali, permettendo al sistema di apprendere le migliori risposte per diverse situazioni.

2.1 Introduzione

Per garantire il funzionamento ottimale del framework Rasa senza incorrere in problemi dovuti a conflitti tra librerie Python, è fondamentale creare un ambiente virtuale dedicato. Questo approccio permette di isolare l'ambiente di sviluppo, evitando interferenze con versioni di librerie già presenti nel sistema e facilitando la gestione delle dipendenze specifiche per Rasa.

2.2 Ambiente Virtuale

Per la creazione dell'ambiente virtuale e dell'intera struttura del progetto abbiamo bisogno di questi elementi:

- **Python 3.10:** Rasa supporta le versioni 3.8, 3.9 e 3.10. Python 3.10 è richiesto per le versioni 3.4.x e successive di Rasa.
- **Anaconda o Miniconda**
- **Visual Studio Code (VSCode)**

Una volta installato Miniconda, possiamo procedere con la creazione di un ambiente virtuale specifico per il progetto Rasa.

1. **Apri Anaconda Prompt** o il terminale di comando.
2. **Crea un nuovo ambiente virtuale** utilizzando Python 3.10:

```
conda create --name rasa_env python=3.10
```

3. **Attiva l'ambiente virtuale:**

```
conda activate rasa_env
```

4. **Installa Rasa e le dipendenze:**

```
pip install rasa pandas fuzzywuzzy python-Levenshtein flask
```

2.3 Database

Il dataset **AC Milan Store Products** contiene informazioni sui prodotti disponibili nello store ufficiale del Milan. Ogni riga rappresenta un prodotto con le relative caratteristiche.

Colonna	Descrizione
Categoria	Indica la macro-categoria del prodotto, ad esempio <i>Kit Da Gara, Allenamento</i> .
Tipologia	Specifica il tipo di prodotto, come <i>Maglia, Pantaloncini, Giacca, Pallone</i> .
Nome	Nome completo del prodotto, comprensivo della stagione di riferimento (es. <i>Milan Home 2024/25</i>).
Disponibilità	Indica se il prodotto è attualmente disponibile (<i>SI</i> o <i>NO</i>).
Sesso	Specifica il target del prodotto: <i>Uomo, Donna, Bambino, Unisex</i> .
Taglie	Elenco delle taglie disponibili (es. <i>XS, S, M, L, XL, XXL</i> oppure <i>6 anni, 8 anni, ...</i> per i bambini).
Colore	Colore principale del prodotto (es. <i>Rosso e Nero</i>).
Prezzo	Prezzo in euro del prodotto.

Il dataset è in formato CSV con separatore ‘;’ e codifica Latin-1. Per la lettura all’interno delle azioni personalizzate è stato utilizzato `pandas` con le opportune operazioni di pulizia e normalizzazione dei dati.

2.4 Struttura del progetto

Dopo aver eseguito il comando `rasa init` per generare la struttura di base, sono stati modificati e aggiunti diversi file per adattare il chatbot alle esigenze specifiche. La struttura finale del progetto è la seguente:

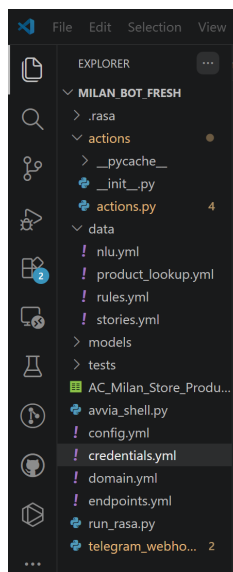


Figura 2.1: Struttura del progetto RASA personalizzata

Di seguito vengono descritti i file principali:

- **actions/actions.py**: contiene le azioni personalizzate che il chatbot esegue in risposta agli intenti. Implementa la logica per cercare prodotti, verificare disponibilità, prezzo, taglie, colore e filtrare per categoria/sexo.
- **data/nlu.yml**: esempi di addestramento per la comprensione del linguaggio naturale, con frasi reali che includono articoli, preposizioni (“da uomo”, “per donna”, “per bambino”) e varianti linguistiche.
- **data/rules.yml**: regole di conversazione che mappano direttamente un intento a un’azione (es. `ask_availability` → `action_check_availability`).
- **data/stories.yml**: storie di addestramento che descrivono conversazioni multi-turno, ad esempio “disponibilità seguita da richiesta prezzo” o “prezzo seguito da taglie”.
- **data/product_lookup.yml**: tabella di lookup con tutti i nomi esatti dei prodotti per migliorare il riconoscimento dell’entità `product_name`.
- **config.yml**: pipeline di elaborazione NLU e politiche di dialogo.
- **domain.yml**: definisce intenti, entità, slot, risposte predefinite e azioni del bot.
- **credentials.yml**: contiene il token del bot Telegram, il canale REST e l’URL del webhook (quando utilizzato).
- **endpoints.yml**: specifica l’URL del server delle azioni personalizzate.
- **telegram_webhook_server.py**: server Flask che funge da ponte tra Telegram e Rasa, risolvendo i problemi di event loop su Windows.
- **AC_Milan_Store_Products.csv**: dataset dei prodotti.

2.5 Logica conversazionale e azioni del chatbot

Il comportamento intelligente del chatbot è affidato a un insieme di **azioni personalizzate** che operano direttamente sul dataset CSV. Quando l’utente formula una richiesta, Rasa NLU classifica l’intento ed estrae le entità (principalmente il nome del prodotto e, quando espresso, il sesso). Il componente Core attiva quindi l’azione corrispondente, che interroga i dati e produce una risposta in linguaggio naturale.

2.5.1 Ricerca dei prodotti

Per rendere il bot tollerante a imprecisioni e varianti linguistiche, la ricerca del prodotto avviene in tre fasi successive:

1. **Corrispondenza esatta**: il nome fornito viene confrontato con i nomi ufficiali presenti nel dataset, ignorando maiuscole e minuscole.
2. **Corrispondenza parziale**: si verifica se la stringa digitata è contenuta in uno dei nomi del catalogo, o viceversa.
3. **Fuzzy matching**: se i passaggi precedenti non danno esito, un algoritmo di similarità (basato su `diffli`) suggerisce il nome più vicino, purché la somiglianza superi una soglia minima. Ciò consente di gestire refusi, abbreviazioni o dimenticanze (ad esempio “Pallone Milan” al posto di *Pallone Milan Essential*).

2.5.2 Risposte e guida alla conversazione

Le azioni del bot non si limitano a restituire un dato puntuale, ma accompagnano l'utente in un dialogo fluido:

- **Disponibilità:** comunica se il prodotto è presente a magazzino e, se l'utente ha indicato un target (uomo, donna, bambino), filtra il risultato di conseguenza.
- **Prezzo:** restituisce il costo del prodotto. Se il prezzo è identico per tutte le varianti, lo comunica direttamente; altrimenti elenca le varianti. Quando il sesso è specificato, il prezzo viene riferito esclusivamente a quella categoria.
- **Taglie:** elenca le misure disponibili, sempre con l'eventuale filtro per sesso.
- **Colore:** fornisce il colore principale del prodotto.
- **Filtri per categoria e sesso:** permette di ottenere l'elenco dei prodotti appartenenti a una determinata categoria (es. *Kit Da Gara* o *Allenamento*) e/o destinati a un particolare target. Dopo la risposta, i filtri vengono azzerati per non condizionare le richieste successive.

Al termine di ogni risposta, il bot propone un **suggerimento contestuale** che invita l'utente a proseguire la conversazione senza dover ripetere il nome del prodotto: ad esempio, dopo aver comunicato la disponibilità, suggerisce *"Ti dico anche il prezzo o le taglie? Scrivi 'prezzo' o 'taglie'."*. In questo modo il dialogo risulta naturale e guidato, simile all'interazione con un commesso in negozio.

Tutte le risposte sono risultate coerenti con il contenuto del dataset e la conversazione è apparsa fluida e naturale, confermando l'efficacia dell'approccio ibrido (Flask + ngrok) e delle strategie di suggerimento contestuale adottate.

3.1 Introduzione

In questo capitolo vengono illustrate tutte le funzionalità del chatbot, fornendo esempi pratici e spiegazioni su come interagire con esso per ottenere informazioni e assistenza. Il chatbot è stato sviluppato per migliorare l'esperienza utente all'interno dello store del Milan, rispondendo rapidamente a domande su prodotti, disponibilità, prezzi, taglie, colori e permettendo di filtrare gli articoli per categoria e target (uomo, donna, bambino). Grazie ai suggerimenti contestuali, il bot guida l'utente passo dopo passo, rendendo la conversazione fluida e naturale.

3.2 Avvio della conversazione

Per iniziare la conversazione con il chatbot, l'utente può utilizzare diverse frasi chiave, come ad esempio *Ciao*, *Buongiorno*, oppure chiedere direttamente informazioni specifiche. Una volta avviata la conversazione, il chatbot si presenta e rimane in attesa di richieste.

3.3 Informazioni sullo store

Il chatbot fornisce tutte le informazioni necessarie sui prodotti nello store. L'utente può chiedere quali categorie sono disponibili o avere una panoramica dell'offerta.

3.4 Prodotti presenti nello store

Una delle funzionalità principali del chatbot è la possibilità di visualizzare i prodotti disponibili. Quando un utente interagisce con il chatbot, può richiedere l'elenco completo dei prodotti oppure filtrarli per tipologia (es. *mostrami le maglie*) o per target (es. *cosa avete per bambino?*).

3.5 Informazioni sui prodotti

Il chatbot offre una funzionalità interattiva che permette agli utenti di ottenere informazioni dettagliate su ogni prodotto. L'utente può chiedere la disponibilità, il prezzo, le taglie e il colore dei vari articoli, anche specificando il target (uomo, donna, bambino). Le principali operazioni supportate sono:

- **Disponibilità:** gli utenti possono chiedere se un determinato prodotto è presente in magazzino. Il bot risponde in modo chiaro, indicando anche se la richiesta è riferita a un target specifico (es. *La maglia Milan Home 2024/25 da uomo è disponibile?*).
- **Prezzo:** gli utenti possono chiedere il prezzo di un prodotto. Il chatbot fornisce il prezzo corrente, con l'eventuale indicazione del target se specificato.
- **Taglie:** per prodotti come maglie, pantaloncini o giacche, l'utente può chiedere quali taglie sono disponibili. Il bot elenca le misure, anche filtrando per sesso.
- **Colore:** l'utente può informarsi sul colore di un articolo. Il bot restituisce il colore principale del prodotto.
- **Filtri per sesso e categoria:** il chatbot consente di visualizzare i prodotti in base al sesso (uomo, donna, bambino) e alla categoria (Kit Da Gara, Allenamento). Questa funzione è utile per esplorare rapidamente il catalogo senza dover ricordare i nomi esatti.

3.6 Riconoscimento robusto dei nomi

Grazie all'uso del **fuzzy matching**, il chatbot è in grado di comprendere il nome del prodotto anche se scritto con piccoli errori di battitura o in modo incompleto. Ad esempio, scrivendo *Pallone Milan* il bot riconosce il *Pallone Milan Essential* e fornisce le informazioni corrette. Questo rende l'interazione più tollerante e adatta a un uso reale.

3.7 Suggerimenti contestuali

Dopo ogni risposta, il chatbot suggerisce all'utente come proseguire la conversazione senza dover ripetere il nome del prodotto. Ad esempio, dopo aver comunicato la disponibilità, il bot chiede *"Ti dico anche il prezzo o le taglie? Scrivi 'prezzo' o 'taglie'."*. In questo modo il dialogo risulta fluido e guidato, simile a quello che si avrebbe con un commesso in un negozio reale.

3.8 Esempio di conversazione completa

La Figura 3.1 mostra un'interazione reale con il chatbot, in cui l'utente richiede dapprima la disponibilità della maglia *Milan Home 2024/25 da uomo*, poi il prezzo, quindi il colore, sfruttando i suggerimenti contestuali senza dover ripetere ogni volta il nome del prodotto.

```
Bot loaded. Type a message and press enter (use '/stop' to exit):
Your input -> Ciao
👋 Ciao! Benvenuto nello store ufficiale del Milan. Come posso aiutarti?
Your input -> La maglia Milan Home 2024/25 da uomo è disponibile?
✅ Il prodotto 'Milan Home 2024/25' è disponibile! (per uomo)
Ti dico anche il prezzo o le taglie? Scrivi 'prezzo' o 'taglie'.
Your input -> Prezzo
💰 Il prezzo di 'Milan Home 2024/25' è **95.00€**.
Vuoi conoscere le taglie? Scrivi 'taglie'.
Your input -> Di che colore?
🎨 Il colore di 'Milan Home 2024/25' è: **Rosso e Nero**.
Posso aiutarti con disponibilità, prezzo o taglie.
Your input ->
```

Figura 3.1: Esempio di dialogo completo: disponibilità, prezzo e colore

Come si può osservare, il chatbot risponde in modo coerente, guida l'utente da una richiesta all'altra e dimostra di saper gestire una conversazione multi-turno mantenendo il contesto. Tutte le risposte sono generate a partire dal dataset CSV e dalle azioni personalizzate implementate nel progetto.

L'ultima parte del progetto consiste nel collegare il chatbot a una piattaforma di messaggistica per renderlo accessibile attraverso un'interfaccia moderna, soprattutto da smartphone. La piattaforma scelta è Telegram, e il servizio `ngrok` è stato utilizzato per esporre il server locale su Internet. A differenza dell'approccio nativo (webhook diretto di Rasa su Telegram), che su Windows presentava instabilità dovute all'*event loop* asincrono, è stata adottata un'architettura a ponte basata su un **server Flask personalizzato**.

4.1 Creazione del Bot Telegram

- Aprire Telegram e cercare **BotFather**.
- Creare un nuovo bot con il comando `/newbot`.
- Assegnare un nome (es. *MilanStoreBot*) e un username (es. *MilanOfficialStoreBot*).
- Ricevere il **token** di accesso, che verrà utilizzato per autenticare le richieste API.

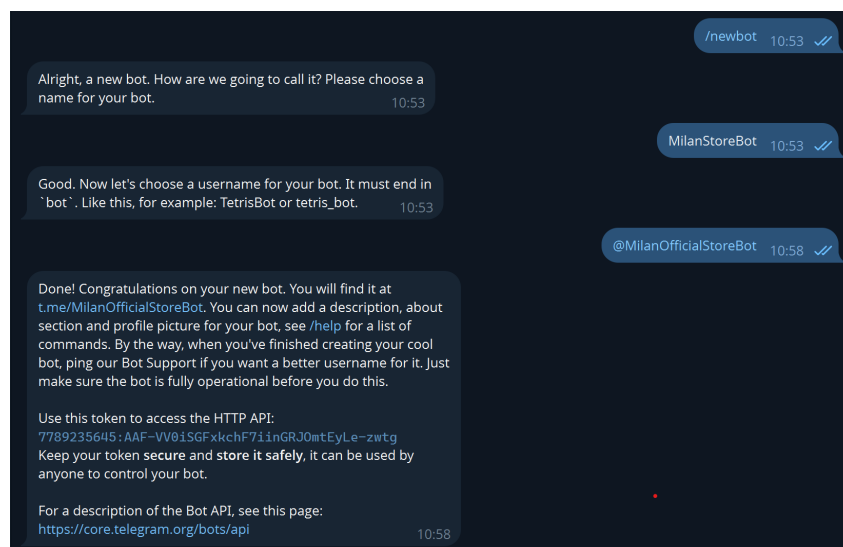


Figura 4.1: Creazione del bot con BotFather

4.2 Architettura di collegamento

Per superare il bug dell'event loop su Windows, il collegamento con Telegram è stato realizzato tramite un **webhook personalizzato** in Flask, che si interpone tra Telegram e Rasa. Il flusso di comunicazione è il seguente:

1. Telegram invia un messaggio al webhook pubblico fornito da ngrok.
2. Ngrok inoltra la richiesta al server Flask in ascolto sulla porta 5000.
3. Flask estrae il testo del messaggio e lo invia a Rasa tramite l'API REST (/webhooks/rest/webhook sulla porta 5005).
4. Rasa elabora la risposta (NLU, Core e azioni) e restituisce i messaggi a Flask.
5. Flask invia ciascun messaggio all'utente Telegram utilizzando il metodo `sendMessage` delle API Telegram.

Questa soluzione ha permesso di mantenere tutti i componenti richiesti dal progetto (webhook, ngrok, Telegram) garantendo piena stabilità su Windows.

4.3 Configurazione del server Flask

Il file `telegram_webhook_server.py` implementa il server webhook e imposta automaticamente il webhook di Telegram all'avvio. Di seguito è mostrato un estratto semplificato del codice:

```
TELEGRAM_TOKEN = "YOUR_TOKEN_HERE"
RASA_URL = "http://localhost:5005/webhooks/rest/webhook"
webhook_url = "https://departed-urgent-borrowing.ngrok-free.dev/webhook"

@app.route("/webhook", methods=["POST"])
def webhook():
    update = request.get_json()
    # Estrae chat_id e testo ...
    # Invia a Rasa e poi risponde all'utente ...
```

Listato 4.1: Estratto del server webhook Flask.

Le credenziali Telegram sono state inserite anche nel file `credentials.yml` di Rasa, limitatamente al canale REST, in modo da abilitare l'endpoint richiesto:

```
telegram:
  access_token: "7789235645:AAF-VV0iSGFxxchF7iinGRJOmtEyLe-zwtg"
  verify: "MilanOfficialStoreBot"

rest:
  # enable REST channel for Flask bridge
```

Listato 4.2: Configurazione in `credentials.yml`.

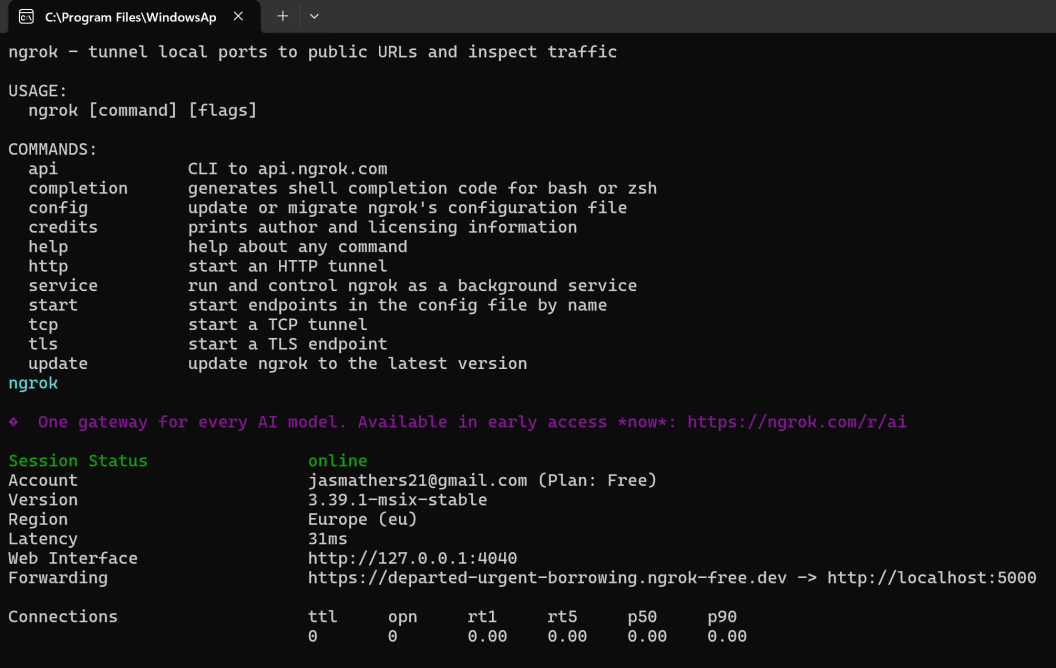
4.4 Avvio del server

Per mettere in funzione il chatbot su Telegram è necessario avviare quattro terminali separati, tutti nella cartella del progetto e con l'ambiente `rasa_env` attivo. L'ordine consigliato è il seguente:

1. **Server delle azioni** (porta 5055): `rasa run actions`
2. **Server API di Rasa** (porta 5005): `rasa run -enable-api -cors "*" "`

3. **Tunnel ngrok** (porta 5000): `ngrok http 5000`
4. **Server webhook Flask** (porta 5000): `python telegram_webhook_server.py`

Il tunnel ngrok fornisce un URL pubblico (nel nostro caso `https://departed-urgent-borrowing.ngrok-free.dev`) che viene automaticamente registrato come webhook del bot Telegram all'avvio del server Flask.



```

C:\Program Files\WindowsAp  x  +  v
ngrok - tunnel local ports to public URLs and inspect traffic

USAGE:
  ngrok [command] [flags]

COMMANDS:
  api           CLI to api.ngrok.com
  completion    generates shell completion code for bash or zsh
  config         update or migrate ngrok's configuration file
  credits        prints author and licensing information
  help          help about any command
  http          start an HTTP tunnel
  service       run and control ngrok as a background service
  start         start endpoints in the config file by name
  tcp          start a TCP tunnel
  tls          start a TLS endpoint
  update        update ngrok to the latest version
ngrok

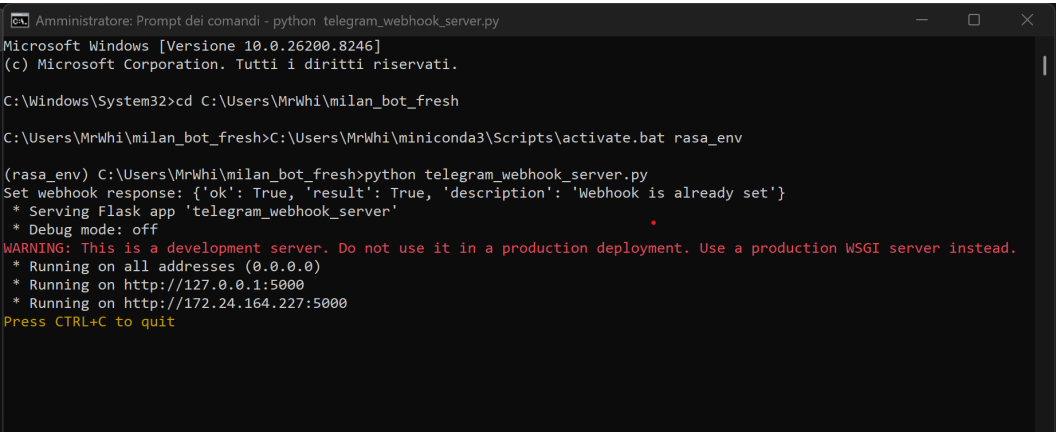
♦ One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             jasmathers21@gmail.com (Plan: Free)
Version             3.39.1-msix-stable
Region              Europe (eu)
Latency             31ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://departed-urgent-borrowing.ngrok-free.dev -> http://localhost:5000

Connections
ttl    opn    rt1    rt5    p50    p90
0      0      0.00  0.00  0.00  0.00

```

Figura 4.2: Avvio di ngrok e ottenimento dell'URL pubblico



```

Amministratore: Prompt dei comandi - python telegram_webhook_server.py
Microsoft Windows [Versione 10.0.26200.8246]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Windows\System32>cd C:\Users\MrWhi\milan_bot_fresh

C:\Users\MrWhi\milan_bot_fresh>C:\Users\MrWhi\miniconda3\Scripts\activate.bat rasa_env
(rasa_env) C:\Users\MrWhi\milan_bot_fresh>python telegram_webhook_server.py
Set webhook response: {'ok': True, 'result': True, 'description': 'Webhook is already set'}
* Serving Flask app 'telegram_webhook_server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.24.164.227:5000
Press CTRL+C to quit

```

Figura 4.3: Avvio del server webhook Flask e registrazione automatica del webhook

4.5 Esempi di interazione

Una volta attivati i quattro terminali, il bot è raggiungibile su Telegram cercando @MilanOfficialSt. Le figure seguenti mostrano alcune richieste tipiche.

4.5.1 Disponibilità di un prodotto

L'utente può chiedere se un articolo è disponibile, anche specificando il target (uomo, donna, bambino). Il bot risponde in modo chiaro e suggerisce la prossima mossa.

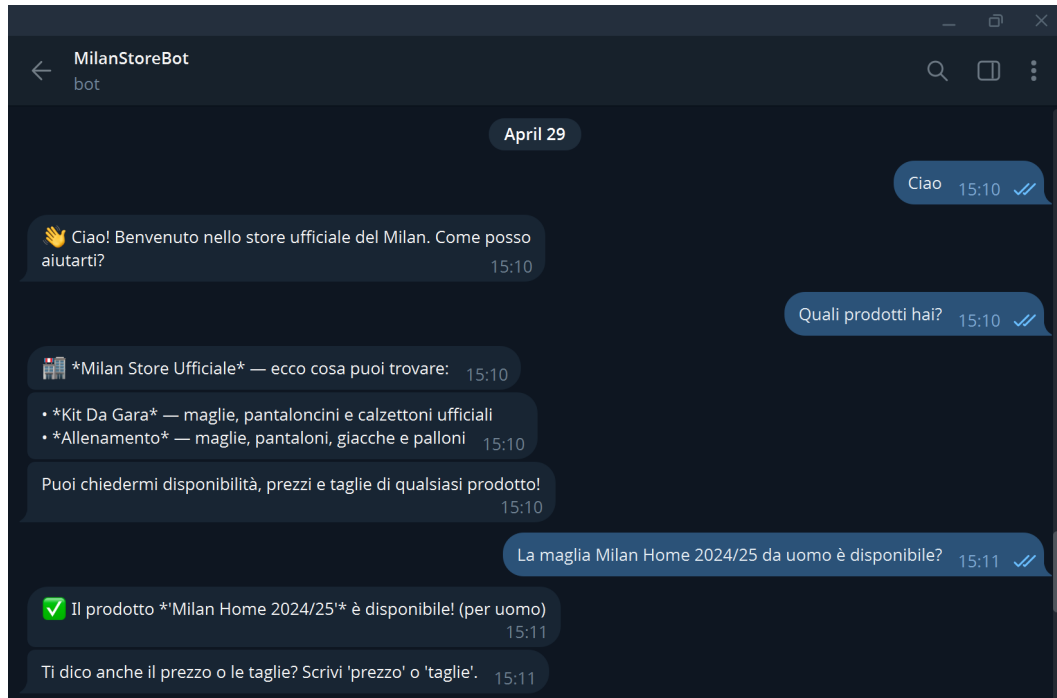


Figura 4.4: Richiesta di disponibilità su Telegram

4.5.2 Prezzo

Dopo aver verificato la disponibilità, è possibile chiedere il prezzo semplicemente scrivendo `prezzo` (il bot mantiene il contesto grazie agli slot interni).

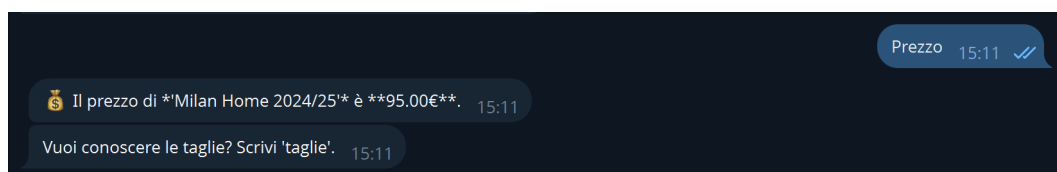


Figura 4.5: Richiesta del prezzo

4.5.3 Taglie e colore

In modo analogo si possono ottenere le taglie disponibili e il colore del prodotto. I suggerimenti contestuali guidano l'utente senza costringerlo a ripetere il nome dell'articolo.



Figura 4.6: Richiesta delle taglie e del colore

4.5.4 Indisponibilità di un prodotto

Il bot gestisce anche il caso in cui un articolo non sia presente in magazzino. Ad esempio, richiedendo il *Pallone Milan Essential*, il sistema risponde con un messaggio chiaro e fornisce comunque suggerimenti per proseguire la conversazione (figura 4.7).



Figura 4.7: Richiesta di un prodotto non disponibile

4.5.5 Filtro per categoria e target

È possibile filtrare i prodotti per sesso o categoria. L'utente può, ad esempio, chiedere “*Che articoli ci sono per bambino?*” e ottenere l'elenco completo degli articoli destinati a quella fascia (figura 4.8). La stessa funzionalità è disponibile per le categorie *Kit Da Gara* e *Allenamento*.

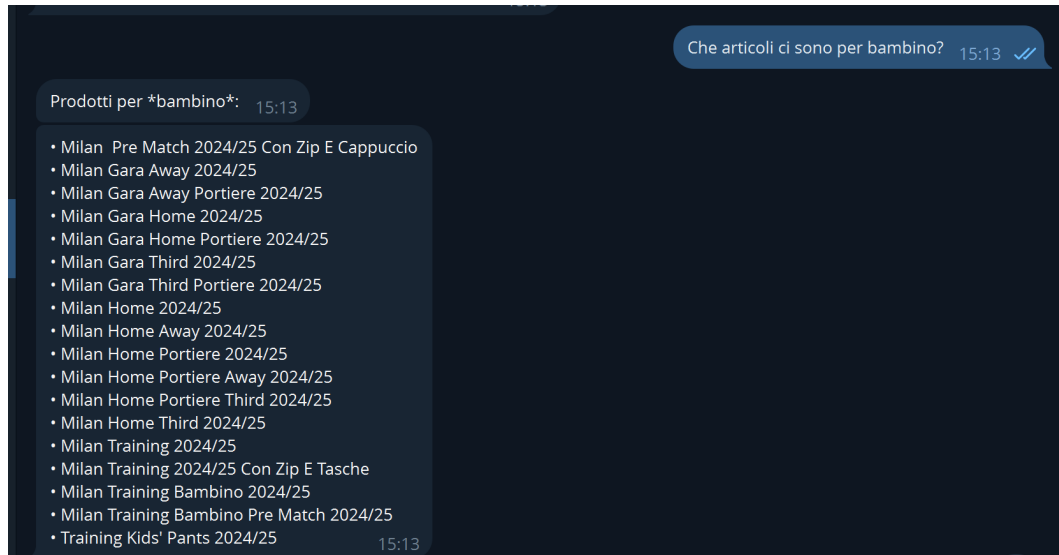


Figura 4.8: Filtro

4.6 Considerazioni finali

L'architettura ibrida Flask + ngrok + Rasa ha consentito di realizzare un assistente conversazionale stabile e reattivo su Telegram, nonostante le limitazioni del webhook nativo su Windows. Il bot è in grado di sostenere conversazioni multi-turno, comprendere richieste in linguaggio naturale con varianti linguistiche e fornire informazioni aggiornate in tempo reale, dimostrando l'efficacia dell'approccio adottato.

Bibliografia

Rasa Documentation,

<https://rasa.com/docs/>

Ngrok Documentation,

<https://ngrok.com/docs>

Flask Documentation,

<https://flask.palletsprojects.com/>

Telegram Bot API,

<https://core.telegram.org/bots/api>